Midterm Project Report
CMPSC 448
Cameron Chilson

1. Introduction and Problem Definition

The goal of this project was to investigate whether machine learning could identify which LLM generated a particular response. I implemented two neural networks, a Convolutional Neural Network (CNN) and a Long Short-Term Memory (LSTM) network, to classify responses from ChatGPT, Gemini, and Qwen.

The project investigated four research questions:
RQ1: Can a neural network identify an LLM using only its generated response?
RQ2: Does including the original prompt improve classification accuracy?
RQ3: Can LLM-identifying patterns generalize across different task categories?
RQ4: What linguistic or structural characteristics distinguish different LLMs?

2. Dataset Collection and Preparation

I created a dataset consisting of 100 prompts divided into five categories of general knowledge, mathematics, coding, creative writing, and reasoning. Each prompt was submitted to ChatGPT, Gemini, and Qwen, giving me 300 total responses, with 100 responses from each model. Responses were collected automatically using Python and the OpenAI, Google Gemini, and Groq APIs. The models I used were gpt-5-mini, gemini-3.5-flash-lite, and qwen/qwen3.8-27b.

The collection script stored the model name, original prompt, generated response, and task category in a spreadsheet that was exported to CSV. Data collection involved several challenges, including unavailable model endpoints, rate limits, and empty responses. These were addressed by changing providers, adjusting request settings, and adding retries.

Before training, the text was converted to lowercase, tokenized, and transformed into numerical sequences. A vocabulary of 8,798 tokens was constructed from training data. Each sequence was padded or truncated to 300 tokens, and an embedding layer converted tokens into 64-dimensional vectors. The dataset was divided into 70% training, 15% validation, and 15% testing. Responses were split by prompt ID to make sure that different models' responses to the same question did not appear in separate subsets.

3. CNN and LSTM Implementation

Two neural networks were implemented using PyTorch. The CNN consisted of an embedding layer, a one-dimensional convolutional layer with 64 filters and a kernel size of 3, ReLU activation, global max pooling, and a fully connected layer with three output classes. The CNN was designed to recognize local word and formatting patterns associated with each LLM. The LSTM contained an embedding layer, an LSTM layer with 64 hidden units, and a fully connected classification layer. Unlike the CNN, the LSTM processed tokens sequentially to learn patterns across the text.

Both models used the Adam optimizer, cross-entropy loss, a learning rate of 0.001, and a batch size of 16. Training was performed for up to 12 epochs, with the best model weights selected using validation macro F1. A random seed of 42 was used. Each network was evaluated using three configurations: input only, output only, and combined input and output.

4. Results and Research Questions

The results of the experiments for RQ1 and RQ2 are shown in the table below:

Model	Input Type	Accuracy	Macro F1
CNN	Input Only	33.33%	0.327
CNN	Output Only	64.44%	0.633
CNN	Input and Output	71.11%	0.698
LSTM	Input Only	33.33%	0.305
LSTM	Output Only	46.67%	0.459
LSTM	Input and Output	44.44%	0.426

For RQ1, both networks performed above the 33.33% chance baseline when using generated responses. The CNN outperformed the LSTM, achieving 64.44% accuracy compared

with 46.67%. The confusion matrices showed that ChatGPT was easiest to identify. The output-only CNN correctly classified 14 of 15 ChatGPT responses, compared with 8 of 15 Qwen responses and 7 of 15 Gemini responses. Qwen and Gemini were frequently confused.

For RQ2, both networks achieved 33.33% accuracy using only the original prompt, which was expected because all three LLMs received identical questions. Adding the prompt to the generated response improved CNN accuracy from 64.44% to 71.11%, but decreased LSTM accuracy from 46.67% to 44.44%. Therefore, prompt context helped the CNN but did not benefit the LSTM in these experiments.

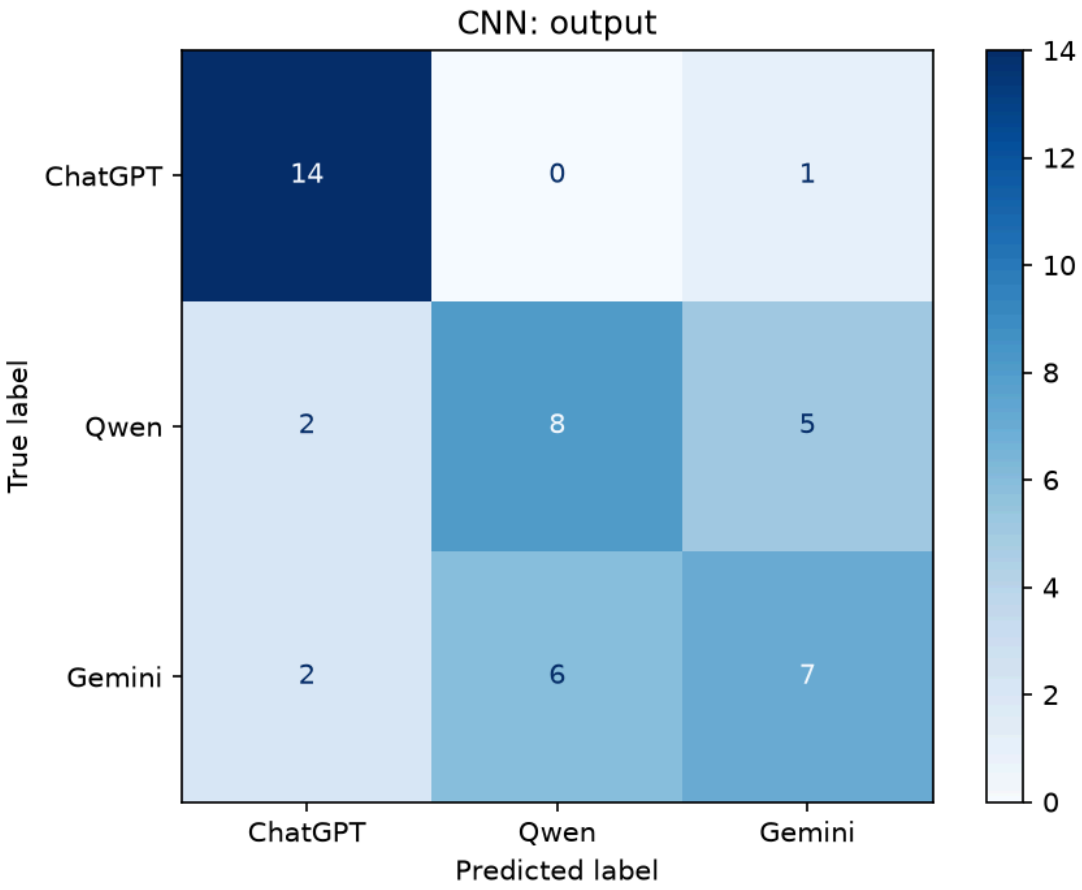


Figure 1: CNN Confusion Matrix Using Generated Responses

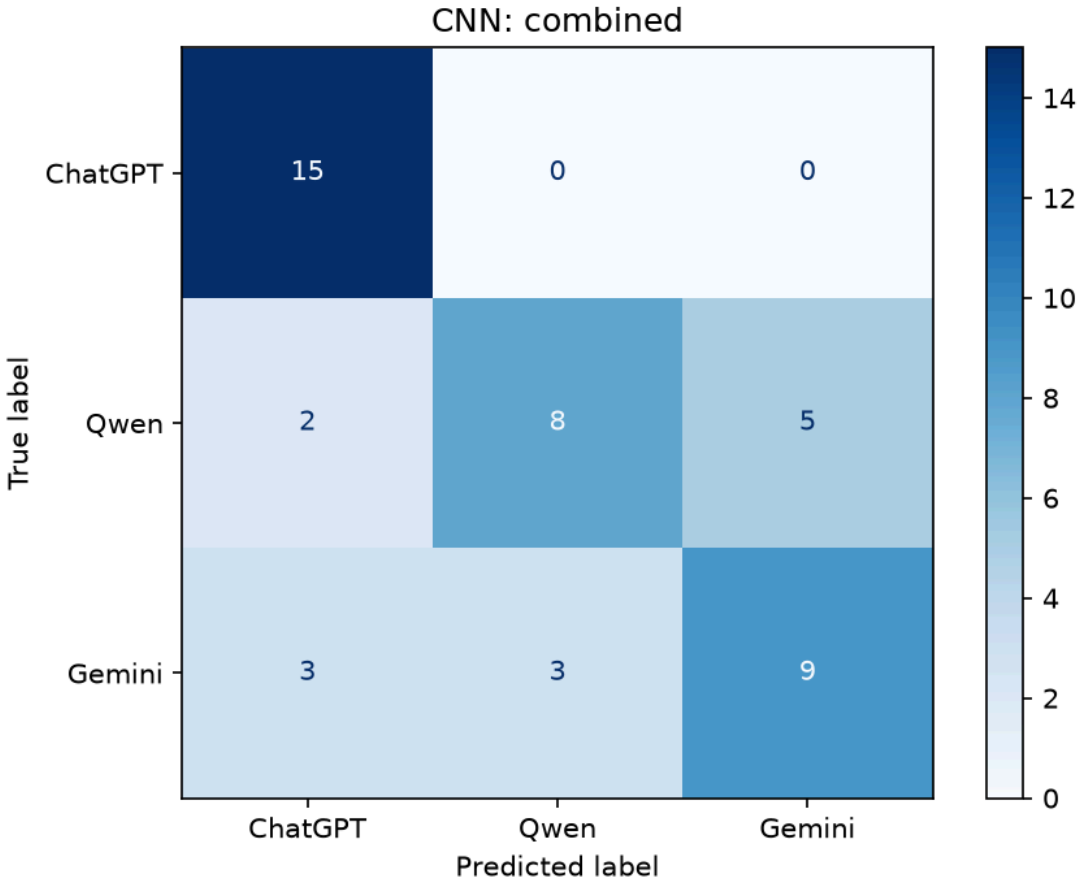


Figure 2: CNN Confusion Matrix Using Prompts and Responses

For RQ3, I trained new CNN and LSTM classifiers using general knowledge, mathematics, coding, and reasoning responses. Creative-writing responses were excluded from training and used as an unseen test domain. The training and validation sets contained 192 and 48 responses, respectively, while the creative-writing test set contained 60 responses.

Model	**Accuracy**	**Macro F1**
CNN	43.33%	0.375
LSTM	36.67%	0.239

Both models performed worse than in their original output-only experiments. The CNN decreased from 64.44% to 43.33%, while the LSTM decreased from 46.67% to 36.67%. These results suggest that some LLM-identifying patterns transferred across categories, but the

classifiers also relied on task-specific characteristics. The experiment demonstrates that identifying LLMs becomes more challenging when testing on unfamiliar types of writing.

For RQ4, I analyzed response length, vocabulary diversity, and formatting across all 300 responses.

Characteristic	**ChatGPT**	**Gemini**	**Qwen**
Average Words	332.48	401.40	400.84
Unique-Word Ratio	0.561	0.501	0.487
Responses using Lists	60%	76%	68%
Responses with Headings	5%	73%	69%

ChatGPT produced shorter responses on average and had the highest unique-word ratio. It also used Markdown headings far less frequently than Gemini and Qwen. Gemini and Qwen were more similar in response length and formatting. These similarities may help explain why the classifiers frequently confused their responses.

The results suggest that response length, vocabulary, and formatting may act as distinguishing characteristics. However, further experiments would be required to determine whether these features directly influenced classification accuracy. The unique-word ratio is also affected by response length.

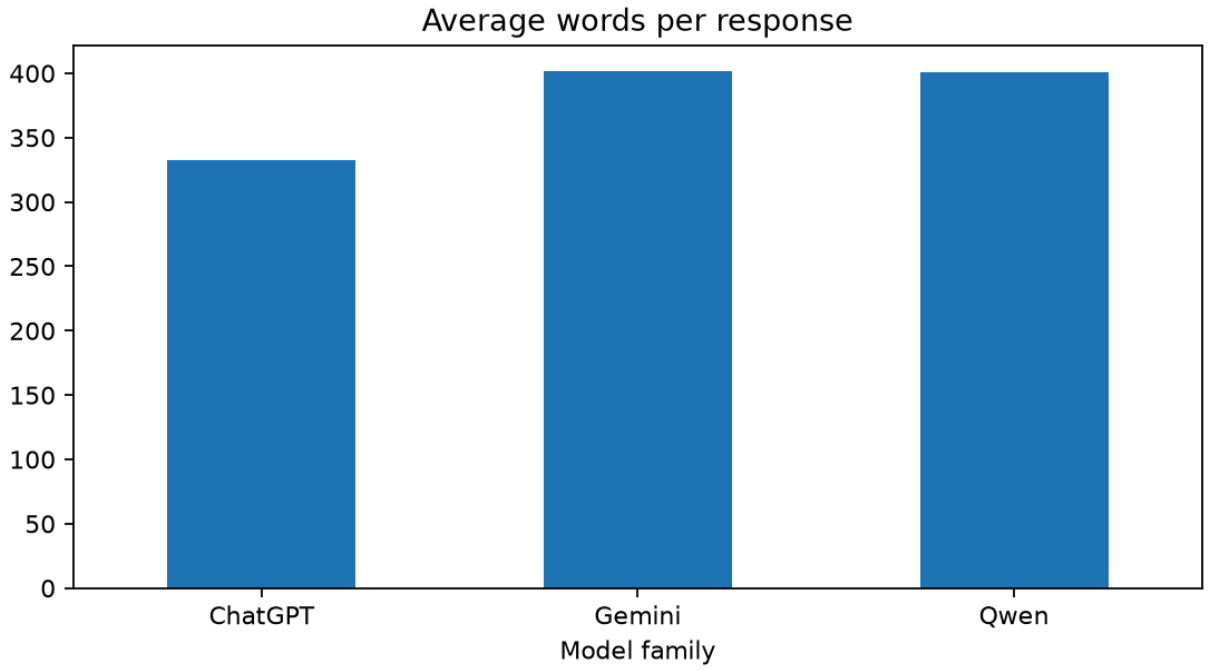


Figure 3: Average Response Length by LLM

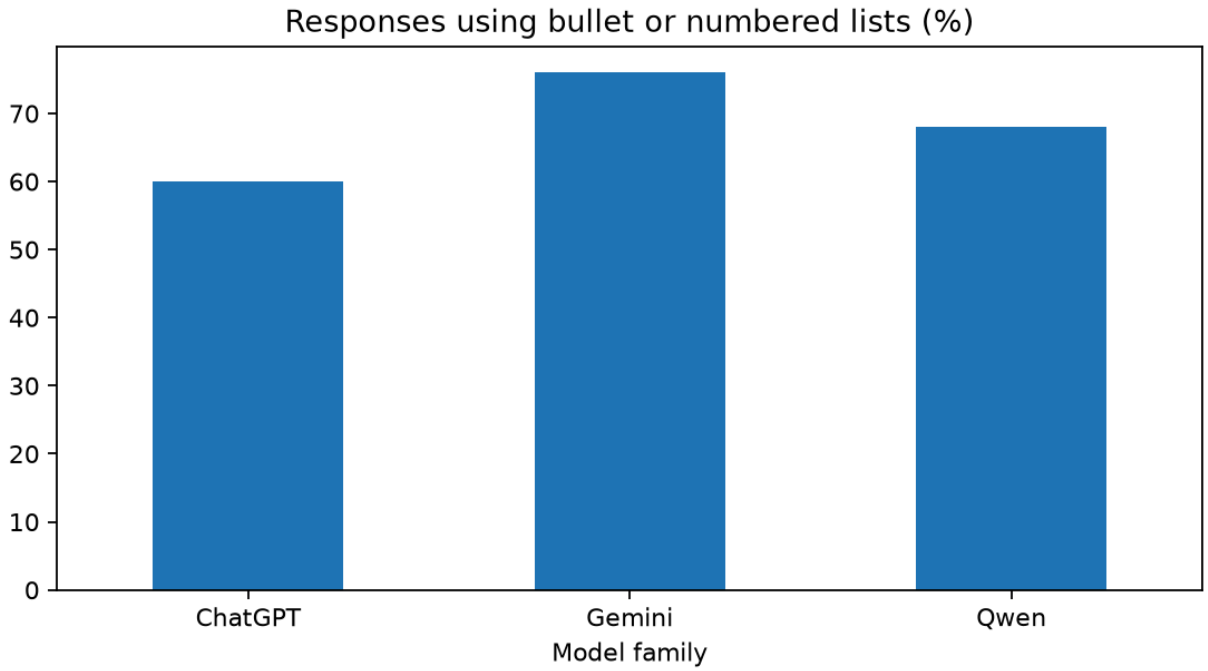


Figure 4: Percentage of Responses Containing Lists

5. Discussions and Limitations

The CNN consistently outperformed the LSTM when classifying generated responses. One possible explanation is that the CNN was more effective at recognizing local word combinations and formatting patterns, while the LSTM struggled to learn reliable differences from the limited dataset. The confusion matrices and linguistic analysis suggest that ChatGPT had more distinctive writing characteristics, whereas Gemini and Qwen shared more similarities.

The cross-domain experiment also showed that classification accuracy decreased substantially when testing on unseen creative-writing prompts, indicating limitations in how well learned LLM fingerprints generalized.

Several limitations should be considered. The dataset contained only 300 responses, and the test set for the main experiments included just 45 examples. Additionally, text sequences were truncated to 300 tokens, potentially removing useful information. Some API generation settings were adjusted during collection, which may have introduced variation in responses. Future improvements could include larger datasets, additional models, repeated train/test splits, and experiments that remove formatting features to determine their impact on classification.

6. Lessons Learned and Conclusion

One of the biggest challenges I faced during this project was collecting responses from the three LLMs. I initially planned to use Claude and Llama, but ran into issues with API costs and unavailable models. I eventually switched to Qwen using Groq. I also encountered rate limits and empty responses, which required me to modify my Python script and retry requests. This helped me understand how important it is to handle errors properly when collecting data automatically.

Implementing the CNN and LSTM also helped me understand how different neural network architectures approach text classification. The experiments emphasized the importance of preventing data leakage, selecting appropriate evaluation metrics, and interpreting confusion matrices rather than relying only on accuracy.

Overall, the project demonstrated that neural networks can identify meaningful patterns in LLM-generated text. The best-performing model was the CNN using combined prompts and responses, which achieved **71.11% accuracy**, compared with a 33.33% chance baseline. However, distinguishing different LLMs remains challenging, particularly when models produce similar responses or when classifiers encounter unfamiliar tasks.